

Introduction to COOL & Compiler

1. COOL Language

COOL = Classroom Object-Oriented Language

- COOL is an **object-oriented programming language** designed mainly for **teaching compiler construction**.
- It provides important modern programming concepts:
 - **Abstraction**
 - **Static typing**
 - **Inheritance**
 - **Automatic memory management**
- COOL programs are compiled into **MIPS assembly language**.
- COOL source files generally use the **.cl extension**.

2. COOL Compiler

The course project builds a complete compiler that translates:

COOL Source Code → MIPS Assembly Code

The generated MIPS assembly can be executed using a MIPS simulator such as **SPIM**.

Important:

Source Language: COOL

Target Language: MIPS Assembly

3. Major Phases of Compiler

The lecture divides the compiler into **four major phases**:

1. Lexical Analysis

Converts the source program into **tokens**.

Example:

x = 10;

Tokens include:

identifier → x

operator → =

number → 10

symbol → ;

2. Parsing / Syntax Analysis

Checks whether the sequence of tokens follows the **grammar/syntax** of the language.

It produces a structural representation such as a **parse tree / syntax tree**.

3. Semantic Analysis

Checks the **meaning** of the program.

Examples:

- Type compatibility
- Valid declarations
- Valid method return type
- Inheritance/type relationships

4. Code Generation

Converts the checked program into **target code**, here:

MIPS Assembly

★ **GATE Shortcut**

Lexical → Tokens
Parsing → Structure
Semantic → Meaning/Types
Code Generation → Target Code

4. Optimization

Optimization can be performed to make the generated code **more efficient**.

However, in this lecture/project, optimization is described as **optional**.

So remember:

Optimization ≠ mandatory phase in the four phases listed in this lecture.

5. Basic Structure of a COOL Program

Every COOL program contains a class named:

Main

The Main class must contain a method:

main()

The main method takes **no arguments**.

Basic structure:

```
class Main {  
    main() : Int {  
        ...  
    };  
};
```

Important syntax:

- class → keyword
- Main → class name
- { } → class body
- main → main method
- : → return type declaration
- ; → terminates declarations/methods

6. COOL is an Expression Language

One of the important concepts in the lecture:

COOL is an expression-oriented language.

A method does not require an explicit return statement.

The **value of the method body becomes the value returned by the method**.

For example, conceptually:

```
main() : Int {  
    10  
};
```

The body evaluates to 10, so the method's value is 10.

7. Method Return Type

A method declares its return type using **:**.

Example:

```
main() : Int
```

Here:

- main → method name
- () → no parameters
- : → return type separator
- Int → declared return type

The compiler checks whether the **actual/inferred type of the body conforms to the declared return type**.

Example

If:

```
main() : Int {  
    l.out_string("Hello");  
};
```

and out_string produces an IO value, then:

IO $\neq$ Int

So the compiler reports a **type/semantic error**.

Changing the return type to a compatible type resolves the problem.

★ GATE Point

This kind of error belongs to:

Semantic Analysis / Type Checking

—not lexical analysis or parsing.

8. Expression Blocks

A block contains multiple expressions enclosed in { }.

Example:

```
{  
    expression1;  
    expression2;  
    expression3;  
}
```

Expressions are evaluated **in order**.

Most important rule:

The value of a block is the value of its LAST expression.

Example:

```
{  
    5;  
    10;  
    20;  
}
```

The value of the block is:

20

★ GATE Shortcut

Block value = Last expression's value

9. Semicolon

Expressions inside a block are separated/terminated using semicolons.

Example:

```
{  
    x;  
    y;  
    z;  
}
```

Forgetting the required semicolon can cause a **syntax error**.

Therefore:

Missing semicolon → Syntax/Parsing problem

10. Attribute vs Object

This is an important conceptual distinction.

Suppose:

i : IO;

This declares an **attribute/field** named `i` whose type is `IO`.
But it **does not create an IO object**.

To create an object:

```
i <- new IO;
```

Remember:

Declaration ≠ Object Creation

- Declaration → specifies name and type
- new → allocates/creates an object

11. new Operator

The new operation is used to **create an object of a class**.

Example:

```
new IO
```

creates an object belonging to class `IO`.

GATE Concept

If a variable/attribute is declared but no object is created, the declaration alone does **not** allocate the intended class object.

12. Input/Output using IO

COOL provides an `IO` class for input/output operations.

For example, an `IO` object can be used to output a string:

```
l.out_string("hello world");
```

The lecture also demonstrates creating an `IO` object and using it for output.

13. Class and Inheritance

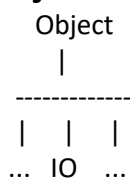
COOL supports **inheritance**.

A class can inherit features from another class.

The important root class is:

`Object`

Object hierarchy



Every other COOL class is ultimately a subclass of `Object`.

★ GATE Point

Object = Root of the COOL class hierarchy

14. Inheritance and IO

Instead of separately creating an `IO` object, a class can inherit from `IO`.

For example, conceptually:

```
class Main inherits IO {
```

```
  ...
};
```

Then `Main` gets the functionality provided through `IO`.

15. self

`self` refers to the **current object**.

It is conceptually similar to this in languages such as Java/C++.

Example:

self.method()

means:

Call method() on the current object.

16. Implicit Method Dispatch

A method can be called without explicitly writing the receiver.

For example:

method()

is interpreted as a call on:

self

Conceptually:

method()

≈

self.method()

17. Compiler Error Classification — GATE Focus

Problem	Compiler Phase
Invalid character/token	Lexical Analysis
Missing semicolon / invalid syntax	Parsing
Type mismatch	Semantic Analysis
Wrong method return type	Semantic Analysis
Generate MIPS code	Code Generation
Improve generated code efficiency	Optimization

Easy Trick

$L \rightarrow P \rightarrow S \rightarrow C$

Lexical → Parsing → Semantic → Code Generation

★ GATE Quick Revision

1. **COOL** = Classroom Object-Oriented Language.
2. COOL is designed for **compiler construction education**.
3. COOL source file → **.cl**
4. COOL compiler → **MIPS assembly**
5. MIPS simulator mentioned → **SPIM**
6. Every COOL program has class → **Main**
7. Main class contains → **main()**
8. COOL is an → **Expression language**
9. No explicit return is required for the method body.
10. **Block value = value of last expression.**
11. **:** is used to specify a **method return type**.
12. Type mismatch → **Semantic Analysis**.
13. Missing semicolon → **Syntax/Parsing error**.
14. Declaration of an attribute does **not** create its object.
15. **new** → **object creation/allocation**.
16. Object → **root of class hierarchy**.
17. **self** → **current object**.
18. Method call without receiver → implicitly uses **self**.
19. Four major compiler phases in this lecture:
Lexical Analysis → Parsing → Semantic Analysis → Code Generation
20. **Optimization is optional** in the project description.

💧 One-line GATE Memory Trick

COOL → .cl → Lexical → Parsing → Semantic → Code Generation → MIPS (.s) → SPIM
This is the set I would revise before attempting GATE-style questions from this lecture.

COOL Programming — Factorial Example

1. Basic COOL Program Structure

Every COOL program must contain:

- A class named **Main**
- A method named **main** inside Main
- The main method does not take arguments.

Example structure:

```
class Main {  
    main() : Object {  
        ...  
    };  
};
```

Important

Every method declaration in the class body is terminated by a **semicolon ;**.

GATE Trap:

Missing ; after a method → **Syntax error**.

2. Input/Output in COOL

To perform I/O, an **IO object** is required.

Important methods discussed:

Method	Purpose
--------	---------

out_string()	Prints a string
--------------	-----------------

in_string()	Reads a string
-------------	----------------

Example:

```
io.out_string("Hello");
```

To read:

```
io.in_string()
```

The lecture uses an IO object to read input and print output.

3. String → Integer Conversion

Input obtained using in_string() is a **String**.

For factorial, we need an **Integer**.

COOL provides the A2I class:

A2I = ASCII to Integer

It provides methods for conversion between **strings and integers**.

Conceptually:

String

↓ A2I

Integer

↓ I2A

String

Important methods

- A2I → String → Integer
- I2A → Integer → String

The lecture uses these conversions before and after performing the factorial computation.

★ GATE Point

If input is obtained through `in_string()`, it is a **String**, so arithmetic operations require conversion to an integer.

4. Using a Library Class

If a program uses another class such as `A2I`, the class definition must be supplied to the compiler.

Multiple COOL class files can be provided together to the compiler.

Conceptually:

```
coolc Main.cl A2I.cl
```

The compiler treats the supplied class definitions as part of the same program.

GATE Concept

Library class must be available to the compiler when the program depends on it.

Otherwise, the compiler can report an **undefined class** error.

5. Method Definition

A COOL method can have:

- Method name
- Parameters
- Parameter types
- Return type
- Body

Example:

```
fact(i : Int) : Int {
```

```
  ...
```

```
};
```

Here:

- `fact` → method name
- `i` → parameter
- `Int` → parameter type
- `Int` → return type

6. Method Calls and self

If a method is called without an explicit object/receiver, the call is dispatched to **self**.

Example:

```
A2I()
```

can conceptually mean a call on:

`self`

If the current class has inherited the required method, it can invoke it.

Remember

No explicit receiver → self

This is especially important when understanding calls to inherited methods.

7. Recursive Factorial

Factorial is defined recursively as:

$$fact(0) = 1$$

and

$$fact(n) = n \times fact(n - 1), n > 0$$

In COOL, this can be represented using an if-then-else expression.

Conceptually:

```

if i = 0 then
  1
else
  i * fact(i - 1)
fi

```

Example

$$3! = 3 \times 2 \times 1 = 6$$

$$6! = 720$$

The lecture implements factorial recursively first.

8. if-then-else

COOL uses an expression of the form:

```

if condition then
  expression1
else
  expression2
fi

```

Important:

if in COOL is an expression, so it produces a value.

For factorial:

```

if i = 0 then
  1
else
  i * fact(i - 1)
fi

```

The selected branch determines the value of the expression.

9. Iterative Factorial

The recursive implementation can be rewritten using a **loop**.

We need an accumulator:

```
fact : Int <- 1;
```

Initially:

```
fact = 1
```

Then repeatedly multiply by i until i becomes zero.

Algorithm:

```
fact = 1
```

```
while i != 0 loop
```

```
  fact = fact * i
```

```
  i = i - 1
```

```
pool
```

Finally:

```
fact
```

is the result.

10. let Expression

Local variables are declared using a **let expression**.

Conceptually:

```
let fact : Int <- 1 in
```

```
  ...
```

A let has two important parts:

1. Declaration

The variable and its type/initialization.

2. Body

The expression in which the variable is available.

The lecture also notes that multiple variables can be declared in a let.

★ GATE Point

The variable declared by let is available within the **body of that let**.

11. let and Scope

Example:

```
let x : Int <- 10 in
```

```
  x + 5
```

Here x is available inside the body.

Conceptually:

Declaration

↓

Scope

↓

Body

The body determines the resulting value of the let expression.

12. while Loop

COOL uses:

while condition loop

body

pool

Important keywords:

- while → starts the loop
- loop → starts loop body
- pool → ends the loop

Example:

```
while i != 0 loop
```

```
  ...
```

```
pool
```

13. Body of while

A key point from the lecture:

The body of a while loop is a **single expression**.

If we need multiple operations, we use a **block** as that expression.

Example:

```
while i != 0 loop
```

```
{
```

```
  fact <- fact * i;
```

```
  i <- i - 1;
```

```
}
```

```
pool
```

So:

while

↓

condition

↓

single expression

↓

block

↓

multiple statements

14. Block Expression

A block can contain multiple expressions:

```
{  
  expr1;  
  expr2;  
  expr3;  
}
```

They are evaluated sequentially.

Most important rule:

The value of the block is the value of its last expression.

Example:

```
{  
  fact <- fact * i;  
  i <- i - 1;  
  fact;  
}
```

The value of this block is the value of:

fact

because it is the last expression.

15. Assignment Operator — VERY IMPORTANT ⚠

COOL does **NOT** use = for assignment.

Assignment is performed using:

<-

Example:

```
fact <- fact * i;  
i <- i - 1;
```

The same backward arrow is also used for **initialization**.

Example:

```
let fact : Int <- 1 in
```

★ GATE Trap

Do NOT confuse:

x <- 5

with:

x = 5

In COOL:

- <- → **assignment**
- = → **comparison/equality operator**

16. = Is Equality, NOT Assignment

This is one of the **most important traps in this lecture**.

In COOL:

i = 0

means:

Compare i with 0.

It does **not** assign zero to i.

The result of the comparison is a **Boolean**.

So:

i = 0

has a Boolean result.

17. Why Wrong Assignment Can Cause Infinite Loop

Suppose the programmer accidentally writes something equivalent to:

`i = i - 1;`

thinking = means assignment.

But in COOL, this is a **comparison**.

So `i` is never actually decreased.

Therefore the loop condition:

`i != 0`

may remain true forever.

Result:

Infinite loop

The lecture demonstrates that this mistake can cause the program to keep consuming memory until it runs out.

★ GATE MUST REMEMBER

`<-` → Assignment

`=` → Equality comparison

This is probably the **single most important syntax trap from this lecture**.

18. Recursive vs Iterative Factorial

Recursive

Uses:

`fact(i - 1)`

and the base case:

`fact(0) = 1`

Iterative

Uses:

- let
- accumulator
- while
- assignment

Comparison:

Recursive	Iterative
Uses function calls	Uses loop
Has base case	Has loop condition
<code>fact(i-1)</code>	<code>i <- i-1</code>
Uses call stack	Uses accumulator

19. Complete Conceptual Flow

The factorial program performs:

User Input

↓

`in_string()`

↓

String

↓

A2I

↓

Integer

↓

factorial computation

↓

Integer
 ↓
 I2A
 ↓
 String
 ↓
 out_string()
 ↓
 Output

This is the **overall flow you should remember for the lecture.**

GATE Quick Revision Table

Concept	GATE Point
COOL	Classroom Object-Oriented Language
Input	in_string()
Output	out_string()
String → Integer	A2I
Integer → String	I2A
Assignment	<-
Equality	=
Current object	self
Local variable	let
Loop start	while ... loop
Loop end	pool
Multiple expressions	Block { }
Block value	Last expression
Conditional	if ... then ... else ... fi
Recursive factorial	i * fact(i-1)
Base case	fact(0) = 1
Class containing main	Main
Main method	main()

★ Most Important GATE Traps

1. Assignment vs Equality

x <- 10 → Assignment

x = 10 → Comparison

2. Block value

```
{
  E1;
  E2;
  E3;
}
```

Value = E3

3. while body

while takes **one expression** as its body.

For multiple operations → use a **block**.

4. let

let creates local variables whose scope is its body.

5. Input type

in_string() → **String**, not Integer.

6. Conversion

String --A2I--> Integer --I2A--> String

7. Method call

No explicit receiver → call is on **self**.

8. Method termination

A method declaration in the class body ends with:

;

9. Recursive factorial

fact(0) = 1

fact(n) = n * fact(n-1)

10. Infinite loop trap

Using = instead of <- does **not update the variable**; it performs comparison, potentially causing the loop condition never to become false.



Final GATE Memory Line

<- = Assign | == Compare | let = Local variable | while...loop...pool = Loop | {} = Block |
Block value = Last expression | A2I = String→Int | I2A = Int→String | self = Current object.

COOL: Lists, let, case & Dynamic Types

1. Multiple let Bindings

COOL allows multiple variables to be declared in a single let expression.

General form:

let

 x : Type <- value,

 y : Type <- value,

 z : Type

in

 body

Important syntax

- **Comma ,** separates multiple let bindings.
- A comma is **not placed after the last binding**.
- The variables declared earlier can be used by **subsequent bindings**.

Example:

let

 x : String <- "Hello",

 y : String <- "World",

 z : String <- "\n"

in

 ...



GATE Trap

let bindings → separated by commas

Do NOT confuse this with semicolon termination of other COOL constructs.

2. Sequential Visibility of let Bindings

An important scope rule shown in the lecture:

let

```
x : Int <- 10,  
y : Int <- x + 5
```

in

y

Here y can use x because x was declared earlier.

So the bindings have **sequential visibility**.

GATE Point

If:

```
let x <- ...,  
    y <- x + ...
```

in ...

then x is available when y is defined.

The lecture specifically demonstrates this while constructing the list.

3. Linked List Representation

The lecture builds a **list abstraction**.

Each list node contains two important fields:

item

next

Conceptually:

```
+-----+-----+  
| item | next |  
+-----+-----+
```

For example:

Hello → World → \n → void

Each next points to another list object.

Thus a list node can be viewed as:

List

```
|— item  
|— next → List
```

4. List Initialization

The list has an initialization method that takes:

item

next

Conceptually:

init(i : String, n : List) : List

Inside the method:

item <- i;

next <- n;

self

The method returns:

self — the object being initialized.

Why return self?

It makes **chained initialization calls** convenient.

5. self

self represents the **current object**.

In the list initialization method:

item <- i;

next <- n;

self;
the last expression is self, so the method returns the current list object.

GATE Point

self = reference to the current object.

6. Representing NULL / VOID

COOL does not provide a special literal such as:

NULL

for a null list pointer.

Instead, an **uninitialized variable of a class type has the value void** in the situation demonstrated by the lecture.

Example:

nil : List

with no initializer can be used as the terminating value of the list.

Conceptually:

Hello → World → Newline → void

★ GATE Point

void represents the absence of an object/reference.

7. isvoid

To check whether an object/reference is void, COOL provides:

isvoid expression

Example:

isvoid next

returns a Boolean indicating whether next is void.

The list's flatten() method uses this to detect the end of the list.

8. Recursive List Traversal

The flatten() method recursively traverses the list.

Base case

If:

isvoid next

then the current node is the **last node**.

Return:

item

Recursive case

Otherwise:

item + flatten(next)

Conceptually:

flatten():

if next is void

return item

else

return item concatenated with next.flatten()

9. Example of flatten()

Consider:

Hello → World → \n → void

Then:

flatten>Hello)

works conceptually as:

"Hello" + flatten(World)

```
"Hello" + "World" + flatten(\n)
"Hello" + "World" + "\n"
Final result:
Hello World
This is a classic recursive linked-list traversal.
```

10. Method Dispatch vs Function Notation

A subtle error in the lecture is important.
The programmer initially used a form equivalent to:
flatten(next)
but what was required was **dispatching the flatten method on next**:
next.flatten()

Concept

These are different ideas:

Function call:

```
flatten(next)
```

Method dispatch:

```
next.flatten()
```

The latter means:

Invoke flatten on the object referenced by next.

★ GATE Trap

Always identify whether a call is:

- a normal function invocation, or
- a method dispatch on an object.

11. Generalizing the List

Initially, the list contains only **Strings**.
The lecture then generalizes it so that a list can contain **different types of objects**.
For example:
Hello → World → 42 → ...
Now the item field cannot simply be restricted to String.
The program needs a way to determine the **actual type of an object at runtime**.
This introduces the **case construct**.

12. case Construct

The case construct allows different actions depending on the **runtime type** of an object.
Conceptually:
case item of
 i : Int => ...
 s : String => ...
 o : Object => ...
esac

The exact branch syntax aside, the key idea is:

The runtime type of item determines which branch is selected.

13. case Performs Runtime Type Analysis

Suppose:
item = 42
Then the Int branch can be selected.
If:
item = "Hello"
then the String branch can be selected.

Thus:
Runtime object
↓
Determine runtime type
↓
Select matching case branch

★ GATE Point

case is associated with **runtime type-based selection**.

14. Variable Binding Inside case

A case branch can bind the matched object to a new variable.

Conceptually:

`i : Int => I2A(i)`

If the runtime object is an integer, it is bound to `i` and can then be used as an `Int`.

Similarly:

`s : String => s`

can use the object as a `String`.

Think:

case object of

`variable : Type => expression`

The variable becomes available inside that branch.

15. case and Type-Specific Operations

Suppose a list contains:

`String`

`Int`

To convert every item into a `String` representation:

If item is Int

Use:

`I2A(i)`

If item is String

Use:

`s`

because it is already a string.

Otherwise

The lecture uses `Object` as a general/default branch and calls `abort`.

16. abort

`abort` terminates program execution.

Important point from the lecture:

Although `abort` terminates the program, its **type is `Object`**, not `String`.

This creates a type-checking issue when the surrounding case expression is expected to produce a `String`.

17. Type Checking with Blocks

The lecture solves the previous type issue using a block:

```
{  
  abort();  
  "";  
}
```

The important principle is:

The type/value of a block is determined by its last expression.

Since:

"""

is a String, the block has type String.

Even though abort() occurs first, it terminates execution if reached.

★ GATE Concept

If:

```
{  
  E1;  
  E2;  
  E3;  
}
```

then the block's resulting type/value is determined by **E3**.

18. let vs case Separators ⚠

The transcript explicitly corrects a syntax mistake.

let bindings:

x : Int <- 10,

y : String <- "Hello"

Use:

Comma ,

case branches:

Branches are terminated by:

Semicolon ;

★ VERY IMPORTANT

let → comma-separated bindings

case → semicolon-terminated branches

19. List with Multiple Types

The generalized list can contain:

String

Int

...

Example:

Hello → World → 42 → void

During traversal:

Hello → String branch

World → String branch

42 → Int branch

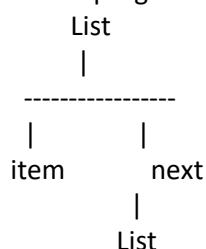
The case expression allows each type to be handled appropriately.

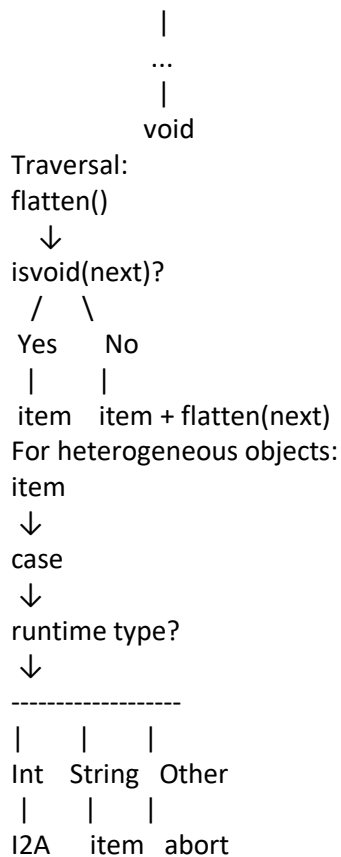
The lecture demonstrates adding 42 to the list and producing:

Hello World 42

20. Complete Conceptual Flow

The final program combines several concepts:





GATE MOST IMPORTANT POINTS

Concept	Remember
Multiple let bindings	Comma-separated
let visibility	Earlier bindings available to later bindings
List node	item + next
List termination	void
Check void	isvoid
Current object	self
Recursive traversal	flatten()
Last list element	isvoid(next) is true
case	Runtime type-based selection
case variable	Bound to object of matching type
abort	Terminates execution; type is Object
Block result	Value/type of last expression
let separator	,
case branch terminator	;
Method on next	next.flatten()
Heterogeneous list	Requires runtime type handling

★ GATE TRAPS FROM THIS LECTURE

Trap 1

```

x : Int <- 10,
y : Int <- x + 5
  
```

☒ y can use x.

Trap 2

let x, y, z

Bindings are separated by:

☒ ,

NOT ;.

Trap 3

isvoid(next) checks:

☒ Whether next is void.

Trap 4

```
{  
  E1;  
  E2;  
  E3;  
}
```

Result:

☒ E3

Trap 5

x <- 10

☒ Assignment

while:

x = 10

☒ Equality comparison.

Trap 6

case determines the branch based on:

☒ Runtime type

Trap 7

abort:

☒ Terminates execution

☒ Has type Object in the context described by the lecture

Trap 8

next.flatten()

means:

☒ Dispatch/call flatten on the object referenced by next.



30-Second Revision

LET

↓

Multiple bindings → comma

↓

Earlier binding visible to later binding

LIST

↓

item + next

↓
last node → next = void
↓
isvoid(next)
↓
flatten recursively

HETEROGENEOUS LIST

↓
case
↓
check runtime type
↓
Int → I2A
String → itself
Other → abort

BLOCK

↓
last expression determines result/type

The highest-value GATE topics from this transcript are:

let scope/order + void/isvoid + recursive list traversal + case runtime typing + block's last-expression rule + comma vs semicolon syntax.

~Anshu